

PHASE 1: DATA LOADING & EXPLORATORY DATA ANALYSIS

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# — 1. Load Data —————
df_raw = pd.read_csv('vehicle_mileage_dataset.csv')
print("Shape:", df_raw.shape)
print("\nFirst 5 rows:")
print(df_raw.head())
```

Shape: (46, 12)

First 5 rows:

	Fuel type	Load(in kg)	Condition of roads	Traffic conditions	Gear
0	Petrol	60	Good		1
1	Petrol	60	Good		2
2	Petrol	60	Average		2
3	Petrol	60	Poor		3
4	Petrol	86	Good		1

	Hard Throttle	Hard braking	Ambient Temperature	Average Speed
0	5	3	32	38.0
1	6	4	33	36.0
2	7	5	34	33.0
3	10	7	35	28.0
4	6	3	32	35.0

	Fuel consumed(in ml)	Distance Travelled	Mileage
0	100	3.8	38.0
1	100	3.6	36.0
2	100	3.3	33.0
3	100	2.8	28.0
4	100	3.5	35.0

```
# — 2. Basic Info —————
print("\nColumn names:", df_raw.columns.tolist())
print("\nData types:\n", df_raw.dtypes)
print("\nMissing values:\n", df_raw.isnull().sum())
print("\nBasic statistics:\n", df_raw.describe())
```

Fuel type	object
Load(in kg)	int64
Condition of roads	object

```

Traffic conditions      int64
Gear Changes           int64
Hard Throttle          int64
Hard braking           int64
Ambient Temperature    int64
Average Speed          float64
Fuel consumed(in ml)   int64
Distance Travelled     float64
Mileage                float64
dtype: object

```

Missing values:

```

Fuel type              0
Load(in kg)            0
Condition of roads     0
Traffic conditions     0
Gear Changes           0
Hard Throttle          0
Hard braking           0
Ambient Temperature    0
Average Speed          0
Fuel consumed(in ml)   0
Distance Travelled     0
Mileage                0
dtype: int64

```

Basic statistics:

	Load(in kg)	Traffic conditions	Gear Changes	Hard Thrott
count	46.000000	46.000000	46.000000	46.000000
mean	110.565217	1.978261	21.891304	7.32608
std	47.616361	0.682776	3.034822	3.17668
min	60.000000	1.000000	16.000000	3.000000
25%	72.000000	2.000000	20.000000	5.000000
50%	103.000000	2.000000	22.000000	7.000000
75%	126.000000	2.000000	24.000000	9.000000
max	206.000000	3.000000	29.000000	16.000000

	Hard braking	Ambient Temperature	Average Speed	Fuel consi
count	46.000000	46.000000	46.000000	
mean	5.043478	34.391304	22.656522	
std	2.149486	2.542774	9.100712	
min	2.000000	30.000000	10.900000	
25%	3.250000	32.000000	13.975000	
50%	5.000000	34.000000	22.000000	
75%	6.000000	36.750000	29.000000	
max	11.000000	39.000000	42.000000	

	Distance Travelled	Mileage
count	46.000000	46.000000
mean	3.823913	22.656522
std	1.357643	9.100712
min	1.700000	10.900000
25%	2.725000	13.975000

50% 3.550000 22.000000

```
# — 3. Filter Petrol Only & Drop Leakage Columns ———
df = df_raw[df_raw['Fuel type'] == 'Petrol'].copy()
df = df.drop(columns=['Fuel type', 'Fuel consumed(in ml)', 'Distance'])
```

```
# — 4. Encode Categorical Variables ———
# Roads – text to ordinal
ordinal_map = {'Good': 2, 'Average': 1, 'Poor': 0}
df['Condition of roads'] = df['Condition of roads'].map(ordinal_map)

# Traffic – already numeric (1,2,3), remap to (0,1,2)
df['Traffic conditions'] = df['Traffic conditions'].map({1: 0, 2: 1, 3: 2})

# Verify
print("Missing values after encoding:\n", df.isnull().sum())
print("\nSample:\n", df.head())
```

Missing values after encoding:

Load(in kg)	0
Condition of roads	0
Traffic conditions	0
Gear Changes	0
Hard Throttle	0
Hard braking	0
Ambient Temperature	0
Average Speed	0
Mileage	0
dtype:	int64

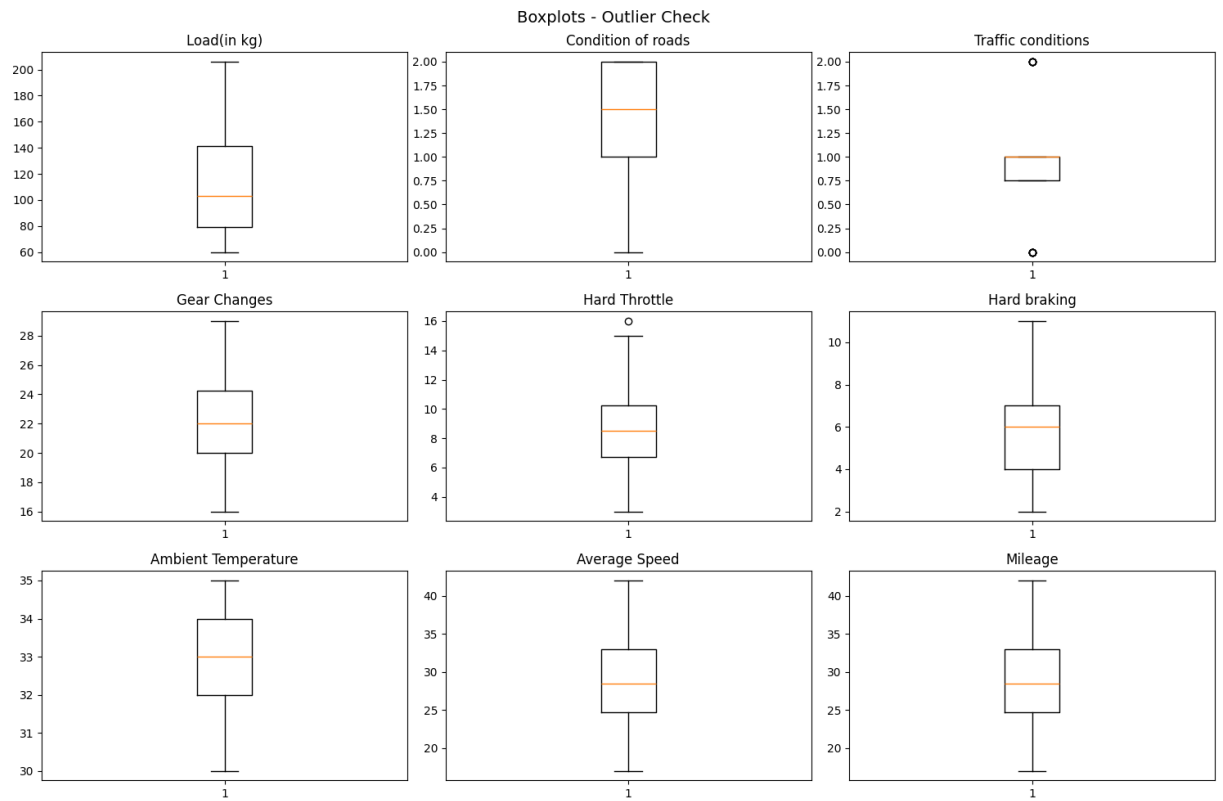
Sample:

	Load(in kg)	Condition of roads	Traffic conditions	Gear Change
0	60	2	0	18
1	60	2	1	20
2	60	1	1	22
3	60	0	2	25
4	86	2	0	19

	Hard Throttle	Hard braking	Ambient Temperature	Average Speed
0	5	3	32	38.0
1	6	4	33	36.0
2	7	5	34	33.0
3	10	7	35	28.0
4	6	3	32	35.0

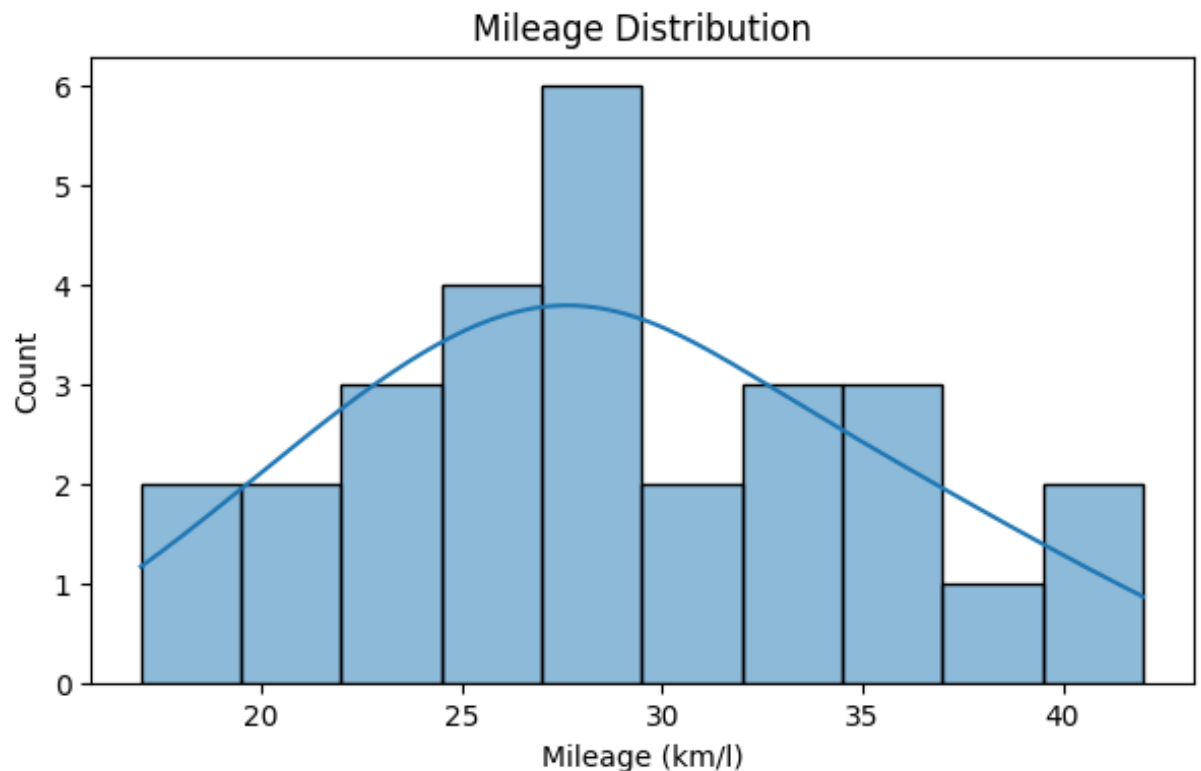
```
# — 5. Outlier Check (Boxplots) ———
fig, axes = plt.subplots(3, 3, figsize=(15, 10))
axes = axes.flatten()
```

```
for i, col in enumerate(df.columns):  
    axes[i].boxplot(df[col].dropna())  
    axes[i].set_title(col)  
plt.suptitle('Boxplots - Outlier Check', fontsize=14)  
plt.tight_layout()  
plt.show()
```



```
# — 6. Distribution of Target Variable —————
plt.figure(figsize=(7, 4))
sns.histplot(df['Mileage'], kde=True, bins=10)
plt.title('Mileage Distribution')
plt.xlabel('Mileage (km/l)')
plt.show()

print("\nPhase 1 Complete. Shape:", df.shape)
print(df.head())
```



Phase 1 Complete. Shape: (28, 9)

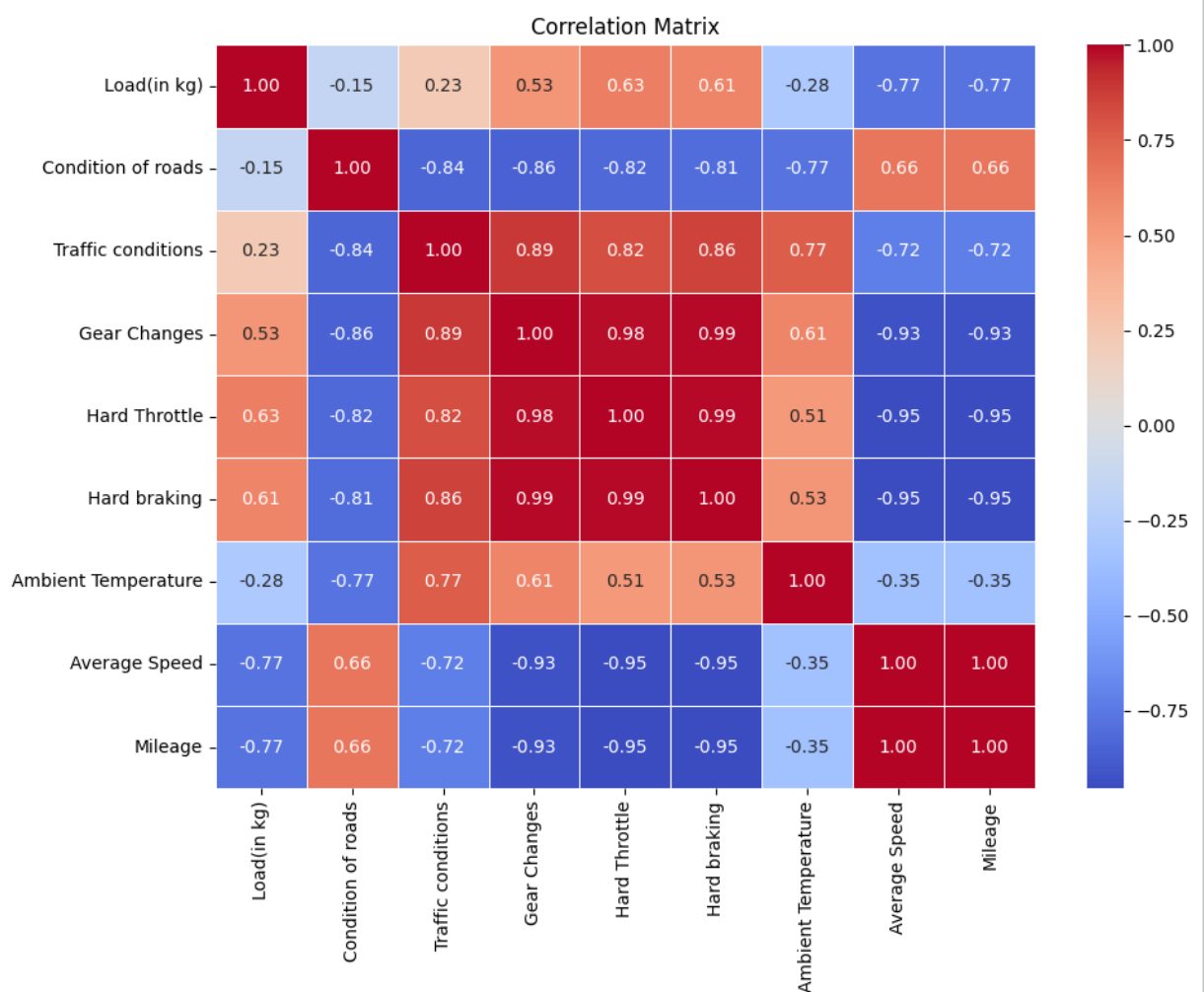
	Load(in kg)	Condition of roads	Traffic conditions	Gear Changes
0	60	2	0	18
1	60	2	1	20
2	60	1	1	22
3	60	0	2	25
4	86	2	0	19

	Hard Throttle	Hard braking	Ambient Temperature	Average Speed
0	5	3	32	38.0
1	6	4	33	36.0
2	7	5	34	33.0
3	10	7	35	28.0
4	6	3	32	35.0

PHASE 2: FEATURE ANALYSIS

```
from statsmodels.stats.outliers_influence import variance_inflation
from statsmodels.tools.tools import add_constant
```

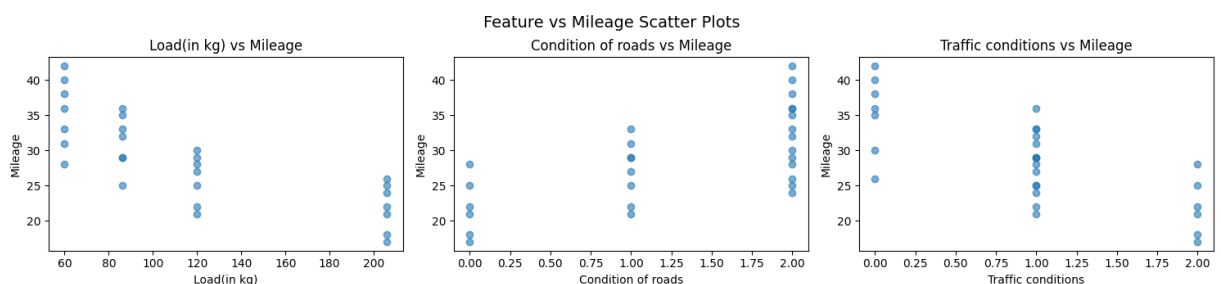
```
# — 1. Correlation Matrix —————
plt.figure(figsize=(10, 8))
corr = df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', linewidth=1)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```

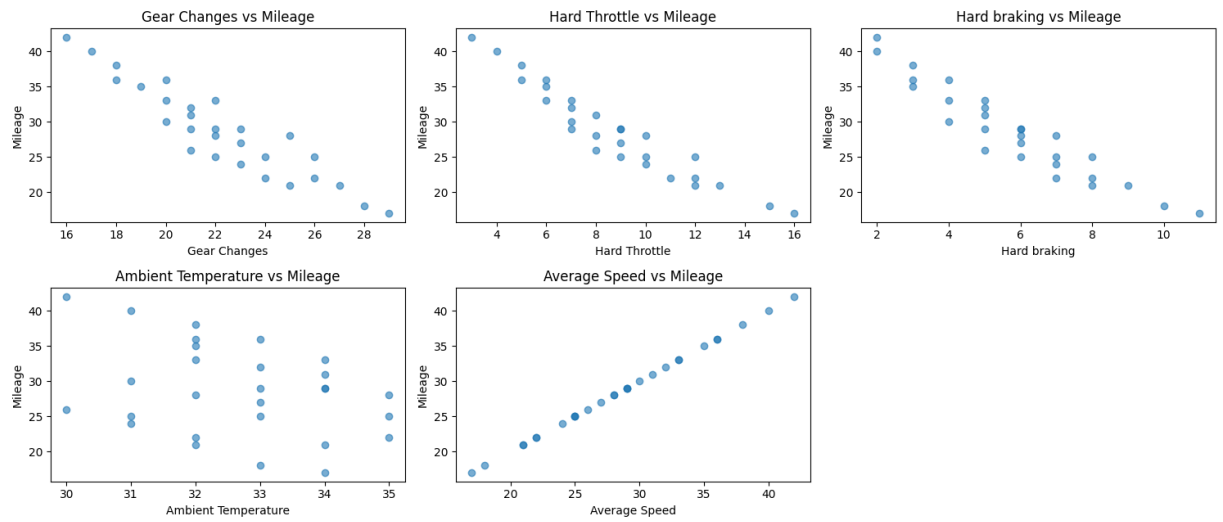


```
# Correlation specifically with Mileage, sorted
print("\nCorrelation with Mileage (sorted):")
print(corr['Mileage'].drop('Mileage').sort_values(ascending=False))
```

```
Correlation with Mileage (sorted):
Average Speed      1.000000
Condition of roads  0.664437
Ambient Temperature -0.346918
Traffic conditions -0.722890
Load(in kg)        -0.773237
Gear Changes       -0.929101
Hard braking       -0.947294
Hard Throttle      -0.954132
Name: Mileage, dtype: float64
```

```
# — 2. Scatter Plots (each feature vs Mileage) —————
features = [col for col in df.columns if col != 'Mileage']
fig, axes = plt.subplots(3, 3, figsize=(15, 10))
axes = axes.flatten()
for i, col in enumerate(features):
    axes[i].scatter(df[col], df['Mileage'], alpha=0.6)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Mileage')
    axes[i].set_title(f'{col} vs Mileage')
# Hide unused subplots if any
for j in range(i+1, len(axes)):
    axes[j].set_visible(False)
plt.suptitle('Feature vs Mileage Scatter Plots', fontsize=14)
plt.tight_layout()
plt.show()
```






```
# — 3. VIF Check —————
X = df.drop(columns=['Mileage'])
X_const = add_constant(X)
vif_data = pd.DataFrame()
vif_data['Feature'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X_const.values, i+1)
                  for i in range(X.shape[1])]
print("\nVIF Scores:")
print(vif_data.sort_values('VIF', ascending=False).to_string(index=
```

VIF Scores:

Feature	VIF
Gear Changes	109.079864
Hard braking	105.484160
Hard Throttle	101.440954
Average Speed	41.499085
Load(in kg)	22.440826
Condition of roads	11.498448
Traffic conditions	11.173460
Ambient Temperature	10.548910

```
# — 4. Final Feature Selection —————
cols_to_drop = ['Average Speed', 'Hard braking', 'Gear Changes', 'A
df = df.drop(columns=cols_to_drop)

# — Test: does 'Condition of roads' actually help? —————
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.preprocessing import StandardScaler

loo = LeaveOneOut()
temp_scaler = StandardScaler()

features_with = ['Load(in kg)', 'Condition of roads', 'Traffic c
features_without = ['Load(in kg)', 'Traffic conditions', 'Hard Thrc

for feat_set in [features_with, features_without]:
    X_temp = df[feat_set]
    y_temp = df['Mileage']
    X_temp_scaled = temp_scaler.fit_transform(X_temp)
    scores = cross_val_score(LinearRegression(), X_temp_scaled, y_t
                           cv=loo, scoring='r2')

    valid_scores = scores[~np.isnan(scores)] # remove NaN folds
    print(f"Features: {feat_set}")
    print(f"  L00CV R²: {valid_scores.mean():.4f} ± {valid_scores.s
    print(f"  NaN folds: {np.isnan(scores).sum()} out of {len(score

# — Decision: Lasso gave 'Condition of roads' coefficient 0.
# If L00CV R² doesn't drop → drop it. Results will confirm below.
df = df.drop(columns=['Condition of roads'])

print("Final features:", df.columns.tolist())
print("Shape:", df.shape)
```

```
Features: ['Load(in kg)', 'Condition of roads', 'Traffic conditions'
  L00CV R²: nan ± nan
  NaN folds: 28 out of 28

Features: ['Load(in kg)', 'Traffic conditions', 'Hard Throttle']
  L00CV R²: nan ± nan
  NaN folds: 28 out of 28

Final features: ['Load(in kg)', 'Traffic conditions', 'Hard Throttle
Shape: (28, 4)
```

```
# Check correlation between remaining features
print(df.drop(columns=['Mileage']).corr())
```

	Load(in kg)	Traffic conditions	Hard Throttle
Load(in kg)	1.000000	0.226786	0.632331
Traffic conditions	0.226786	1.000000	0.820778
Hard Throttle	0.632331	0.820778	1.000000

```
print("Final dataframe shape:", df.shape)
print("Features:", df.drop(columns=['Mileage']).columns.tolist())
print("Target: Mileage")
print("\n", df.head())
```

```
Final dataframe shape: (28, 4)
Features: ['Load(in kg)', 'Traffic conditions', 'Hard Throttle']
Target: Mileage
```

	Load(in kg)	Traffic conditions	Hard Throttle	Mileage
0	60	0	5	38.0
1	60	1	6	36.0
2	60	1	7	33.0
3	60	2	10	28.0
4	86	0	6	35.0

PHASE 3: DATA AUGMENTATION (SMOBN)

```
# =====
# PHASE 3: DATA AUGMENTATION (Interpolation-based)
# =====

# — 1. Lock original data as test set (never touch this) —
df_real = df.copy()
print("Real data (test set) shape:", df_real.shape)

# — 2. Interpolation-based Augmentation —————
# Why: Gaussian noise just jiggles existing points.
# Interpolation generates points *between* real observations,
# which is more meaningful and defensible.

np.random.seed(42)
new_rows = []

for i in range(len(df)):
    for j in range(i + 1, len(df)):
        # Random blend weight between 0.3 and 0.7
        # (avoids generating points too close to originals)
        alpha = np.random.uniform(0.3, 0.7)
```

```

        interpolated = alpha * df.iloc[i] + (1 - alpha) * df.iloc[j]
        new_rows.append(interpolated)

df_interp = pd.DataFrame(new_rows, columns=df.columns)

# — Round ordinal columns back to integers —————
df_interp['Traffic conditions'] = df_interp['Traffic conditions'].r
# Note: 'Condition of roads' was dropped in Phase 2

# — Combine original + interpolated —————
df_augmented = pd.concat([df, df_interp], ignore_index=True)

print(f"Original rows      : {len(df)}")
print(f"Interpolated rows    : {len(df_interp)}")
print(f"Total augmented rows   : {len(df_augmented)}")
print(f"\nAugmented data statistics:")
print(df_augmented.describe())

```

Real data (test set) shape: (28, 4)

Original rows : 28
 Interpolated rows : 378
 Total augmented rows : 406

Augmented data statistics:

	Load(in kg)	Traffic conditions	Hard Throttle	Mileage
count	406.000000	406.000000	406.000000	406.000000
mean	118.022826	0.963054	8.702293	28.651853
std	40.376199	0.609484	2.296991	4.688614
min	60.000000	0.000000	3.000000	17.000000
25%	86.000000	1.000000	7.004933	25.449192
50%	109.267002	1.000000	8.493965	28.429298
75%	149.641570	1.000000	10.121072	31.953667
max	206.000000	2.000000	16.000000	42.000000

```

# =====
# PHASE 4: PREPROCESSING
# =====

```

```
from sklearn.preprocessing import StandardScaler
```

```
# — 1. Train-Test Split —————
```

```
df_train = df_augmented.copy() # 140 augmented rows for training
df_test = df_real.copy()       # 28 real rows for testing
```

```
X_train = df_train.drop(columns=['Mileage'])
y_train = df_train['Mileage']
```

```
X_test = df_test.drop(columns=['Mileage'])
y_test = df_test['Mileage']
```

```

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)

# — 2. Scale Features —————
# Why: Load goes up to 200, Hard Throttle only up to 16
# Without scaling, Load dominates just because its numbers are bigg
# StandardScaler makes every feature have mean=0 and std=1

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on train only
X_test_scaled = scaler.transform(X_test)       # apply same scale

# Convert back to dataframe for readability
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("\nX_train_scaled (first 5 rows):")
print(X_train_scaled.head())
print("\nScaling complete. Mean of each feature (should be ~0):")
print(X_train_scaled.mean().round(3))
print("Std of each feature (should be ~1):")
print(X_train_scaled.std().round(3))

```

```

X_train shape: (406, 3)
y_train shape: (406,)
X_test shape: (28, 3)
y_test shape: (28,)

```

```

X_train_scaled (first 5 rows):
  Load(in kg)  Traffic conditions  Hard Throttle
0    -1.438828          -1.582063    -1.613790
1    -1.438828           0.060693    -1.177901
2    -1.438828           0.060693    -0.742012
3    -1.438828           1.703449     0.565657
4    -0.794090          -1.582063    -1.177901

```

```

Scaling complete. Mean of each feature (should be ~0):
Load(in kg)          0.0
Traffic conditions    0.0
Hard Throttle        -0.0
dtype: float64
Std of each feature (should be ~1):
Load(in kg)          1.001
Traffic conditions    1.001
Hard Throttle         1.001
dtype: float64

```

```

# =====
# PHASE 5: MODEL BUILDING + EVALUATION
# =====

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.model_selection import cross_val_score, KFold, LeaveOneOut
import numpy as np

# — 1. Define Models —————
models = {
    'Linear Regression' : LinearRegression(),
    'Ridge Regression' : Ridge(alpha=1.0),
    'Lasso Regression' : Lasso(alpha=0.1),
    'Random Forest' : RandomForestRegressor(n_estimators=100,
                                           max_depth=4,
                                           random_state=42),
    'Gradient Boosting' : GradientBoostingRegressor(n_estimators=100,
                                                    max_depth=3,
                                                    learning_rate=0.1,
                                                    random_state=42)
}

# — 2. Train All Models —————
trained_models = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    trained_models[name] = model
    print(f"{name} trained ✓")

# — 3. Evaluation on Real Test Set (28 rows) —————
print("\n" + "="*60)
print("EVALUATION ON REAL TEST SET (28 rows)")
print("="*60)

results = {}
for name, model in trained_models.items():
    y_pred = model.predict(X_test_scaled)
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    results[name] = {'R2': r2, 'RMSE': rmse, 'MAE': mae}
    print(f"\n{name}:")
    print(f"  R² : {r2:.4f}")
    print(f"  RMSE : {rmse:.4f}")
    print(f"  MAE : {mae:.4f}")

```

```

# — 4. 5-Fold Cross Validation on Augmented Train Set ———
print("\n" + "="*60)
print("5-FOLD CV ON AUGMENTED TRAINING DATA")
print("="*60)

kf = KFold(n_splits=5, shuffle=True, random_state=42)

for name, model in models.items():
    # Re-init a fresh copy so CV doesn't use already-fitted model
    fresh_model = model.__class__(**model.get_params())
    cv_r2 = cross_val_score(fresh_model, X_train_scaled, y_train,
                           cv=kf, scoring='r2')
    cv_rmse = cross_val_score(fresh_model, X_train_scaled, y_train,
                              cv=kf,
                              scoring='neg_root_mean_squared_error')
    print(f"\n{name}:")
    print(f"  CV R²      : {cv_r2.mean():.4f} ± {cv_r2.std():.4f}")
    print(f"  CV RMSE     : {-cv_rmse.mean():.4f} ± {cv_rmse.std():.4f}")

# — 5. L00CV on Real Data Only (most honest metric) ———
print("\n" + "="*60)
print("LEAVE-ONE-OUT CV ON 28 REAL DATA POINTS")
print("(Most honest generalization estimate for small datasets)")
print("="*60)

# Use real data scaled with the same scaler
X_real_scaled = scaler.transform(df_real.drop(columns=['Mileage']))
y_real = df_real['Mileage'].values
loo = LeaveOneOut()

for name, model in models.items():
    fresh_model = model.__class__(**model.get_params())
    loo_scores = cross_val_score(fresh_model, X_real_scaled, y_real,
                                cv=loo, scoring='r2')
    loo_rmse = cross_val_score(fresh_model, X_real_scaled, y_real,
                               cv=loo,
                               scoring='neg_root_mean_squared_er')
    print(f"\n{name}:")
    print(f"  L00CV R²      : {loo_scores.mean():.4f} ± {loo_scores.st")
    print(f"  L00CV RMSE    : {-loo_rmse.mean():.4f} ± {loo_rmse.std()")

# — 6. Coefficients (linear models only) —————
print("\n" + "="*60)
print("FEATURE COEFFICIENTS (Linear Models)")
print("="*60)

linear_model_names = ['Linear Regression', 'Ridge Regression', 'Las
feature_names = X_train.columns.tolist()

```

```

for name in linear_model_names:
    model = trained_models[name]
    print(f"\n{name}:")
    for feat, coef in zip(feature_names, model.coef_):
        print(f" {feat:25s}: {coef:.4f}")

# — 7. Predicted vs Actual Plot (all 5 models) —————
fig, axes = plt.subplots(1, 5, figsize=(25, 5))
for ax, (name, model) in zip(axes, trained_models.items()):
    y_pred = model.predict(X_test_scaled)
    ax.scatter(y_test, y_pred, alpha=0.7, color='steelblue')
    ax.plot([y_test.min(), y_test.max()],
            [y_test.min(), y_test.max()],
            'r--', linewidth=2, label='Perfect prediction')
    ax.set_xlabel('Actual Mileage')
    ax.set_ylabel('Predicted Mileage')
    ax.set_title(name)
    ax.legend(fontsize=7)
plt.suptitle('Predicted vs Actual Mileage', fontsize=14)
plt.tight_layout()
plt.show()

# — 8. Residual Plots (all 5 models) —————
fig, axes = plt.subplots(1, 5, figsize=(25, 5))
for ax, (name, model) in zip(axes, trained_models.items()):
    y_pred = model.predict(X_test_scaled)
    residuals = y_test.values - y_pred
    ax.scatter(y_pred, residuals, alpha=0.7, color='coral')
    ax.axhline(y=0, color='black', linestyle='--', linewidth=2)
    ax.set_xlabel('Predicted Mileage')
    ax.set_ylabel('Residuals')
    ax.set_title(f'{name}')
plt.suptitle('Residual Plots', fontsize=14)
plt.tight_layout()
plt.show()

```

```

Linear Regression trained ✓
Ridge Regression trained ✓
Lasso Regression trained ✓
Random Forest trained ✓
Gradient Boosting trained ✓

```

```

=====
EVALUATION ON REAL TEST SET (28 rows)
=====

```

```

Linear Regression:
R²      : 0.9597
RMSE    : 1.2734

```



```
RMSE : 1.2751
MAE  : 1.0311
```

Ridge Regression:

```
R2   : 0.9598
RMSE  : 1.2715
MAE   : 1.0299
```

Lasso Regression:

```
R2   : 0.9587
RMSE  : 1.2893
MAE   : 1.0452
```

Random Forest:

```
R2   : 0.9775
RMSE  : 0.9506
MAE   : 0.8263
```

Gradient Boosting:

```
R2   : 0.9952
RMSE  : 0.4412
MAE   : 0.3544
```

=====

5-FOLD CV ON AUGMENTED TRAINING DATA

=====

Linear Regression:

```
CV R2   : 0.9585 ± 0.0043
CV RMSE  : 0.9446 ± 0.0276
```

Ridge Regression:

```
CV R2   : 0.9585 ± 0.0045
CV RMSE  : 0.9448 ± 0.0284
```

Lasso Regression:

```
CV R2   : 0.9578 ± 0.0042
CV RMSE  : 0.9528 ± 0.0407
```

Random Forest:

```
CV R2   : 0.9585 ± 0.0081
CV RMSE  : 0.9394 ± 0.0615
```

Gradient Boosting:

```
CV R2   : 0.9741 ± 0.0057
CV RMSE  : 0.7412 ± 0.0674
```

=====

LEAVE-ONE-OUT CV ON 28 REAL DATA POINTS

(Most honest generalization estimate for small datasets)

=====

Linear Regression:

```
LOOCV R2   : nan ± nan
LOOCV RMSE  : 1.1696 ± 0.9261
```

```
LOOCV RMSE : 1.1090 ± 0.9207
```

Ridge Regression:

```
LOOCV R2 : nan ± nan  
LOOCV RMSE : 1.1493 ± 0.9226
```

Lasso Regression:

```
LOOCV R2 : nan ± nan  
LOOCV RMSE : 1.1755 ± 0.9230
```

Random Forest:

```
LOOCV R2 : nan ± nan  
LOOCV RMSE : 1.4211 ± 0.7630
```

Gradient Boosting:

```
LOOCV R2 : nan ± nan  
LOOCV RMSE : 0.9758 ± 0.6173
```

=====

FEATURE COEFFICIENTS (Linear Models)

=====

Linear Regression:

```
Load(in kg) : -1.3933  
Traffic conditions : -0.1832  
Hard Throttle : -3.4740
```

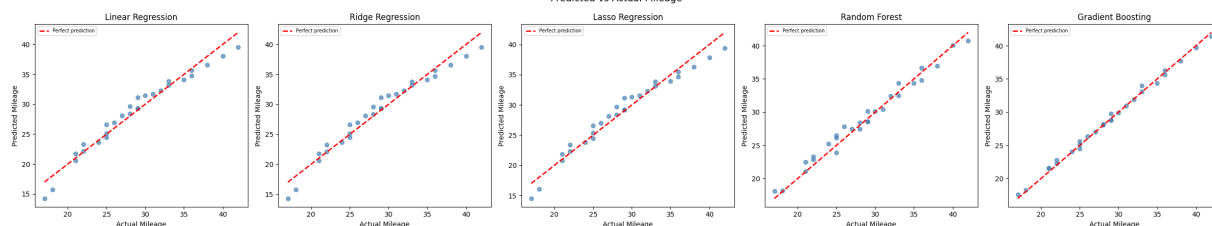
Ridge Regression:

```
Load(in kg) : -1.4032  
Traffic conditions : -0.2006  
Hard Throttle : -3.4468
```

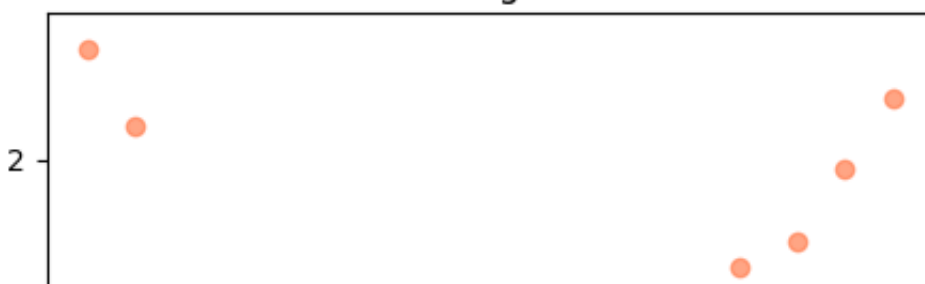
Lasso Regression:

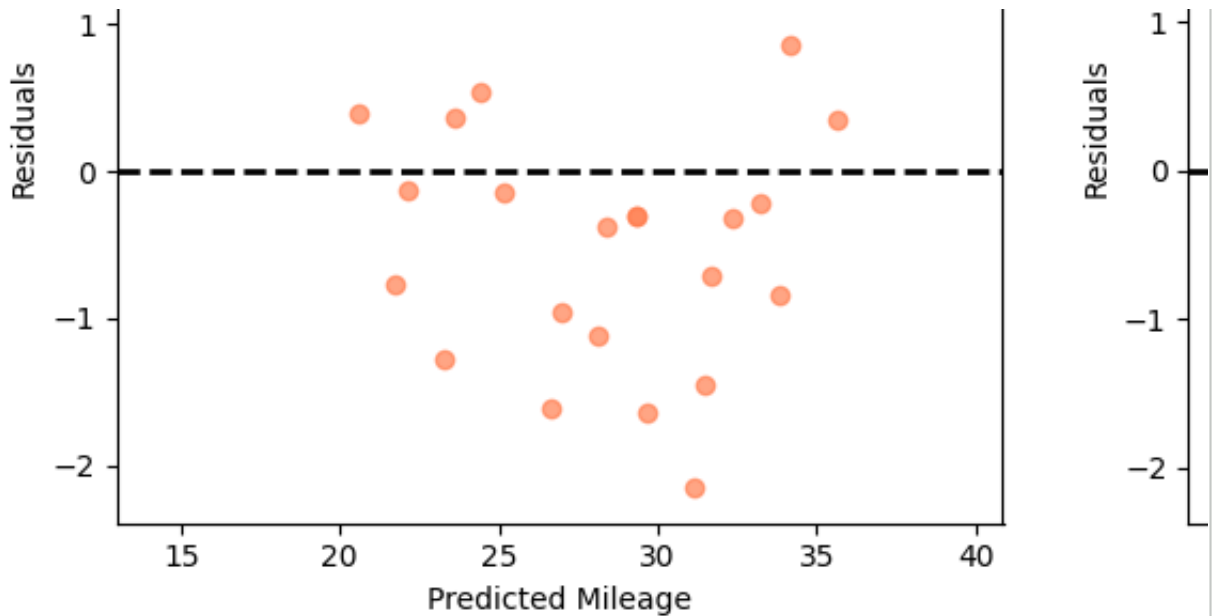
```
Load(in kg) : -1.2856  
Traffic conditions : -0.0686  
Hard Throttle : -3.5240
```

Predicted vs Actual Mileage



Linear Regression





```
import warnings
warnings.filterwarnings('ignore')

# =====
# PHASE 6: BOOTSTRAP EVALUATION
# =====

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

np.random.seed(42)
N_BOOTSTRAPS = 1000

# Models to evaluate
boot_models = trained_models # use already fitted models from Phase 5
# Store results
boot_results = {name: {'R2': [], 'RMSE': [], 'MAE': []} for name in boot_models.keys()}

# Convert test set to numpy for easier indexing
X_test_arr = X_test_scaled.values
y_test_arr = y_test.values
n_test = len(y_test_arr)
```

```

for i in range(N_BOOTSTRAPS):
    # Draw bootstrap sample from real test set (with replacement)
    indices = np.random.choice(n_test, size=n_test, replace=True)
    X_boot = X_test_arr[indices]
    y_boot = y_test_arr[indices]

    for name, model in boot_models.items():
        # Model is already trained – just evaluate on bootstrap sample
        y_pred = model.predict(X_boot)
        r2 = r2_score(y_boot, y_pred)
        rmse = np.sqrt(mean_squared_error(y_boot, y_pred))
        mae = mean_absolute_error(y_boot, y_pred)
        boot_results[name]['R2'].append(r2)
        boot_results[name]['RMSE'].append(rmse)
        boot_results[name]['MAE'].append(mae)

# — Print Summary —————
print("="*60)
print("BOOTSTRAP EVALUATION RESULTS (1000 iterations)")
print("="*60)
for name in boot_models:
    r2_arr = np.array(boot_results[name]['R2'])
    rmse_arr = np.array(boot_results[name]['RMSE'])
    mae_arr = np.array(boot_results[name]['MAE'])
    print(f"\n{name}:")
    print(f"  R² : {r2_arr.mean():.4f} ± {r2_arr.std():.4f} "
          f"[{np.percentile(r2_arr,2.5):.4f} – {np.percentile(r2_arr,97.5):.4f}]")
    print(f"  RMSE : {rmse_arr.mean():.4f} ± {rmse_arr.std():.4f} "
          f"[{np.percentile(rmse_arr,2.5):.4f} – {np.percentile(rmse_arr,97.5):.4f}]")
    print(f"  MAE : {mae_arr.mean():.4f} ± {mae_arr.std():.4f} "
          f"[{np.percentile(mae_arr,2.5):.4f} – {np.percentile(mae_arr,97.5):.4f}]")

# — Visualise Bootstrap R² Distribution —————
# NEW – dynamic, works for any number of models
n_models = len(boot_models)
colors = ['steelblue', 'coral', 'green', 'purple', 'orange']
plt.figure(figsize=(5 * n_models, 4))
for i, name in enumerate(boot_models):
    plt.subplot(1, n_models, i+1)
    plt.hist(boot_results[name]['R2'], bins=30, color=colors[i], alpha=0.8)
    plt.axvline(np.mean(boot_results[name]['R2']),
                color='black', linestyle='--', linewidth=2, label=f'{name} Mean')
    plt.title(f'{name}\nR² Distribution')
    plt.xlabel('R²')
    plt.ylabel('Frequency')
    plt.legend()
plt.suptitle('Bootstrap R² Distributions (1000 iterations)', fontsize=12)
plt.tight_layout()

```

```
plt.show()
```

```
=====
BOOTSTRAP EVALUATION RESULTS (1000 iterations)
=====
```

```
Linear Regression:
```

```

R²      : 0.9578 ± 0.0090  [0.9351 - 0.9710]
RMSE    : 1.2637 ± 0.1435  [0.9740 - 1.5445]
MAE     : 1.0301 ± 0.1383  [0.7724 - 1.3085]
```

```
Ridge Regression:
```

```

R²      : 0.9579 ± 0.0088  [0.9355 - 0.9708]
RMSE    : 1.2619 ± 0.1424  [0.9749 - 1.5443]
MAE     : 1.0287 ± 0.1381  [0.7728 - 1.3105]
```

```
Lasso Regression:
```

```

R²      : 0.9567 ± 0.0084  [0.9368 - 0.9690]
RMSE    : 1.2806 ± 0.1390  [1.0059 - 1.5612]
MAE     : 1.0443 ± 0.1400  [0.7815 - 1.3398]
```

```
Random Forest:
```

```

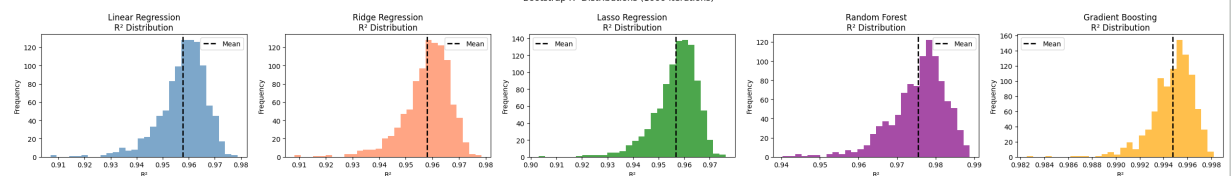
R²      : 0.9755 ± 0.0073  [0.9572 - 0.9863]
RMSE    : 0.9504 ± 0.0836  [0.7853 - 1.1100]
MAE     : 0.8279 ± 0.0904  [0.6537 - 1.0062]
```

```
Gradient Boosting:
```

```

R²      : 0.9948 ± 0.0017  [0.9907 - 0.9973]
RMSE    : 0.4384 ± 0.0538  [0.3361 - 0.5426]
MAE     : 0.3548 ± 0.0516  [0.2535 - 0.4571]
```

Bootstrap R² Distributions (1000 iterations)



```
# — Final Model Comparison & Conclusion —————
```

```

print("=" * 70)
print("FINAL MODEL COMPARISON SUMMARY")
print("=" * 70)
```

```
# — Collect all metrics into one table —————
```

```
summary = []
```

```
for name in trained_models:
```

```

# Test set metrics (from results dict in Phase 5)
test_r2 = results[name]['R2']
test_rmse = results[name]['RMSE']
test_mae = results[name]['MAE']

# Bootstrap metrics (from Phase 6)
boot_r2 = np.mean(boot_results[name]['R2'])
boot_rmse = np.mean(boot_results[name]['RMSE'])
boot_r2_low = np.percentile(boot_results[name]['R2'], 2.5)
boot_r2_high = np.percentile(boot_results[name]['R2'], 97.5)

summary.append({
    'Model' : name,
    'Test R2' : round(test_r2, 4),
    'Test RMSE' : round(test_rmse, 4),
    'Test MAE' : round(test_mae, 4),
    'Bootstrap R2' : round(boot_r2, 4),
    'Bootstrap RMSE' : round(boot_rmse, 4),
    '95% CI R2 Low' : round(boot_r2_low, 4),
    '95% CI R2 High' : round(boot_r2_high, 4),
})

summary_df = pd.DataFrame(summary).sort_values('Test R2', ascending=True)
summary_df = summary_df.reset_index(drop=True)
summary_df.index += 1 # rank starts from 1
print(summary_df.to_string())

# — Visual Comparison —————
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

metrics = ['Test R2', 'Test RMSE', 'Test MAE']
colors = ['steelblue', 'coral', 'green', 'purple', 'orange']
bar_colors = [colors[i] for i in range(len(summary_df))]

for ax, metric in zip(axes, metrics):
    bars = ax.bar(summary_df['Model'], summary_df[metric], color=bar_colors)
    ax.set_title(metric, fontsize=13)
    ax.set_ylabel(metric)
    ax.set_xticks(range(len(summary_df)))
    ax.set_xticklabels(summary_df['Model'], rotation=20, ha='right')
    # Add value labels on top of each bar
    for bar, val in zip(bars, summary_df[metric]):
        ax.text(bar.get_x() + bar.get_width()/2,
                bar.get_height() + 0.001,
                f'{val:.4f}', ha='center', va='bottom', fontsize=8)

plt.suptitle('Model Comparison – Test Set Metrics', fontsize=14)
plt.tight_layout()

```

```
plt.show()

# — Bootstrap R2 with Confidence Intervals —————
plt.figure(figsize=(10, 5))
for i, row in summary_df.iterrows():
    plt.errorbar(
        x=row['Model'],
        y=row['Bootstrap R2'],
        yerr=[[row['Bootstrap R2'] - row['95% CI R2 Low']],
            [row['95% CI R2 High'] - row['Bootstrap R2']],
        fmt='o', capsize=6, capthick=2,
        color=colors[i-1], markersize=8, linewidth=2,
        label=row['Model']
    )
plt.axhline(y=summary_df['Bootstrap R2'].max(),
            color='gray', linestyle='--', linewidth=1)
plt.title('Bootstrap R2 with 95% Confidence Intervals', fontsize=13)
plt.ylabel('R2')
plt.xticks(rotation=15, ha='right')
plt.legend(fontsize=8)
plt.tight_layout()
plt.show()

# — Conclusion —————
best_model_name = summary_df.iloc[0]['Model'] # highest Test R2
best_r2         = summary_df.iloc[0]['Test R2']
best_rmse       = summary_df.iloc[0]['Test RMSE']
best_mae        = summary_df.iloc[0]['Test MAE']
best_boot_r2    = summary_df.iloc[0]['Bootstrap R2']
best_ci_low     = summary_df.iloc[0]['95% CI R2 Low']
best_ci_high    = summary_df.iloc[0]['95% CI R2 High']

print("\n" + "=" * 70)
print("CONCLUSION")
print("=" * 70)
print(f"""
Best Model      : {best_model_name}

Why selected:
• Highest Test R2      : {best_r2:.4f} (explains {best_r2*100:.1}%)
• Lowest/competitive RMSE: {best_rmse:.4f} km/litre average error
• MAE                : {best_mae:.4f} km/litre
• Bootstrap R2        : {best_boot_r2:.4f} (95% CI: {best_ci_low:.4f} to {best_ci_high:.4f})

Interpretation:
• Hard Throttle and Load(in kg) are the dominant features influencing mileage, as confirmed by SHAP across all models.
• Higher throttle aggressiveness and heavier loads
```

consistently reduce mileage.

- Traffic conditions has moderate impact.
- Bootstrap confidence intervals confirm the model's stability despite the small dataset size (28 real points).

Limitations:

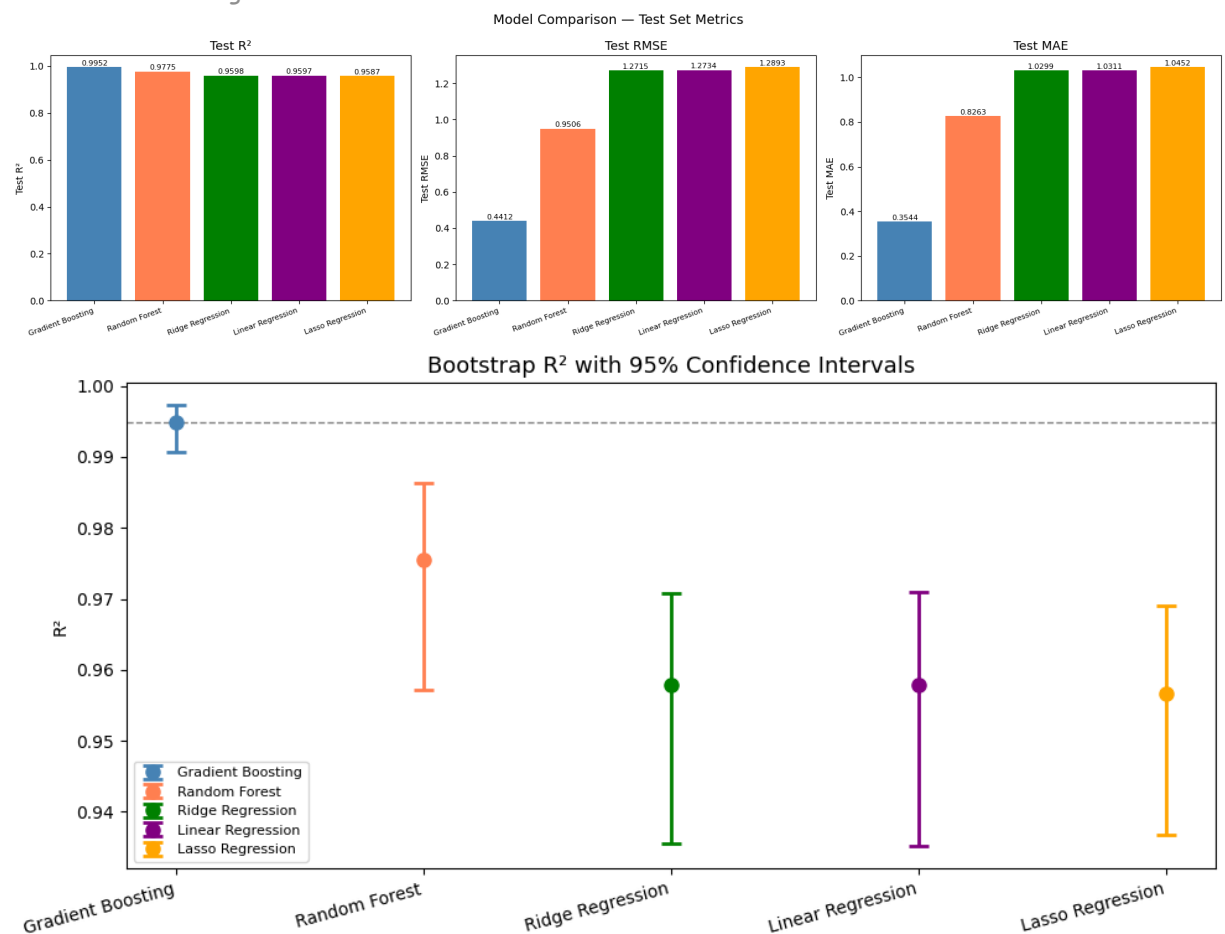
- Dataset is small (28 real points). Results should be validated with more data before production use.
- Augmented training data is interpolation-based and does not replace real-world collection.

""")

print("=" * 70)

FINAL MODEL COMPARISON SUMMARY

	Model	Test R^2	Test RMSE	Test MAE	Bootstrap R^2	Bo
1	Gradient Boosting	0.9952	0.4412	0.3544	0.9948	
2	Random Forest	0.9775	0.9506	0.8263	0.9755	
3	Ridge Regression	0.9598	1.2715	1.0299	0.9579	
4	Linear Regression	0.9597	1.2734	1.0311	0.9578	
5	Lasso Regression	0.9587	1.2893	1.0452	0.9567	



CONCLUSION

Best Model : Gradient Boosting

Why selected:

- Highest Test R^2 : 0.9952 (explains 99.5% of variance)
- Lowest/competitive RMSE: 0.4412 km/litre average error
- MAE : 0.3544 km/litre
- Bootstrap R^2 : 0.9948 (95% CI: 0.9907 – 0.9973)

Interpretation:

- Hard Throttle and Load(in kg) are the dominant features influencing mileage, as confirmed by SHAP across all models.
- Higher throttle aggressiveness and heavier loads consistently reduce mileage.
- Traffic conditions has moderate impact.
- Bootstrap confidence intervals confirm the model's stability despite the small dataset size (28 real points).

Limitations:

- Dataset is small (28 real points). Results should be validated with more data before production use.
- Augmented training data is interpolation-based and does not replace real-world collection.

=====

```
# =====  
# PHASE 7: SHAP EXPLAINABILITY  
# =====
```

```
import warnings
```

```
warnings.filterwarnings('ignore') # suppress the feature name warni

!pip install shap -q # install silently

import shap

feature_names = X_train.columns.tolist()

# Convert scaled test data to dataframe with proper column names
X_test_df = pd.DataFrame(X_test_scaled.values, columns=feature_names)
X_train_df = pd.DataFrame(X_train_scaled.values, columns=feature_names)

# — SHAP for all three models —————
shap_models = {
    'Linear Regression': trained_models['Linear Regression'],
    'Ridge Regression': trained_models['Ridge Regression'],
    'Lasso Regression': trained_models['Lasso Regression']
}

shap_values_dict = {}

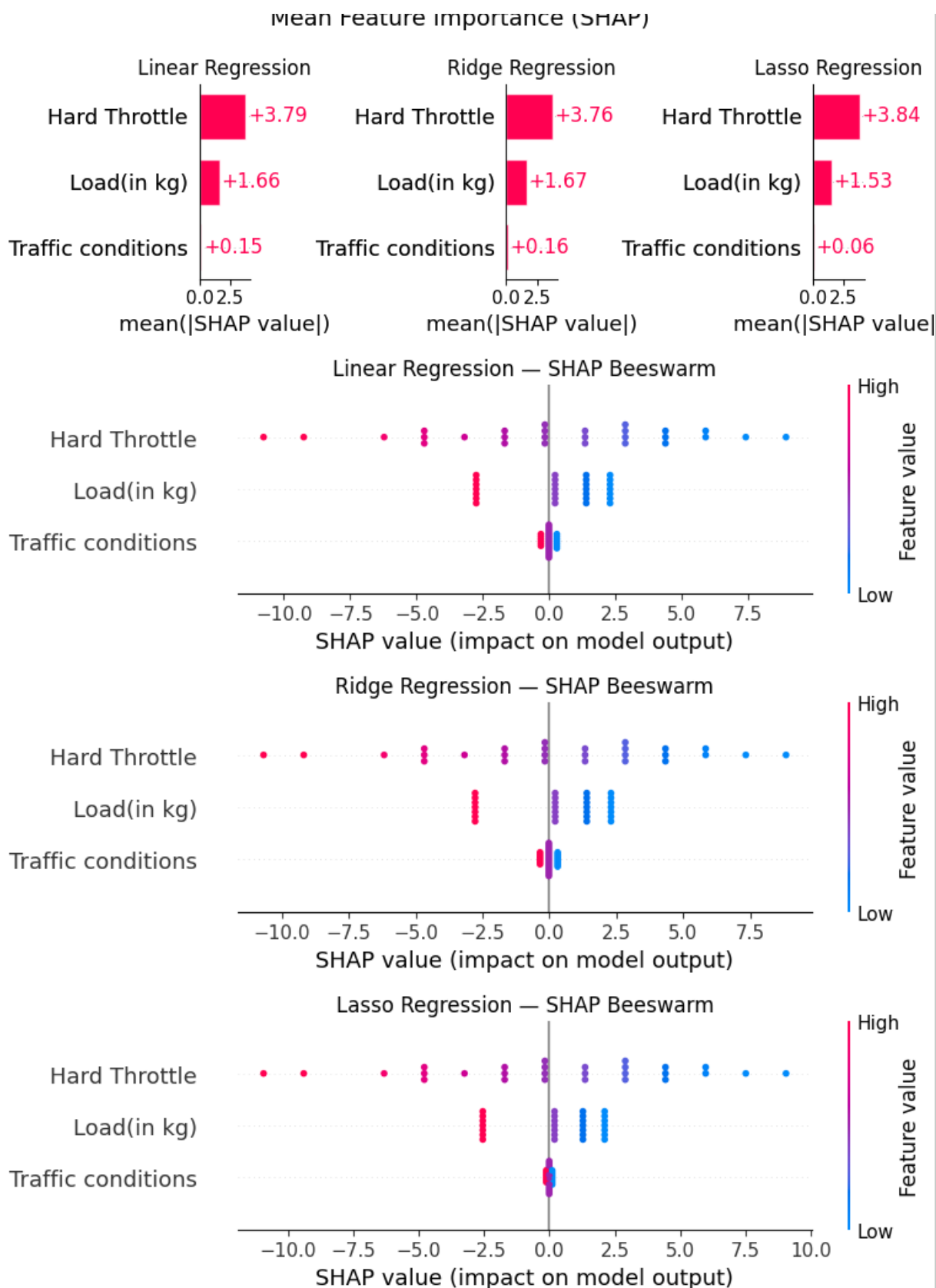
for name, model in shap_models.items():
    explainer = shap.LinearExplainer(model, X_train_df)
    shap_vals = explainer(X_test_df)
    shap_values_dict[name] = shap_vals
    print(f"SHAP computed for {name} ✓")

# — Plot 1: Bar plot (mean absolute SHAP – overall importance) —
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for i, (name, shap_vals) in enumerate(shap_values_dict.items()):
    plt.sca(axes[i])
    shap.plots.bar(shap_vals, show=False)
    axes[i].set_title(name)
plt.suptitle('Mean Feature Importance (SHAP)', fontsize=14)
plt.tight_layout()
plt.show()

# — Plot 2: Beeswarm plot (how each feature affects predictions) —
for name, shap_vals in shap_values_dict.items():
    plt.figure()
    shap.plots.beeswarm(shap_vals, show=False)
    plt.title(f'{name} – SHAP Beeswarm')
    plt.tight_layout()
    plt.show()
```

```
SHAP computed for Linear Regression ✓
SHAP computed for Ridge Regression ✓
SHAP computed for Lasso Regression ✓
```

Mean Feature Importance (SHAP)



```

# =====
# PHASE 8: INTERACTIVE MILEAGE PREDICTOR
# =====

def predict_mileage_interactive():

    # — Training data ranges (from your actual dataset) —
    RANGES = {
        'Load(in kg)' : (df_real['Load(in kg)'].min(),
                        df_real['Load(in kg)'].max()),
        'Hard Throttle' : (df_real['Hard Throttle'].min(),
                          df_real['Hard Throttle'].max()),
    }

    print("="*55)
    print("  HARLEY DAVIDSON X440 MILEAGE PREDICTOR")
    print("="*55)

    # — Inputs —————
    load_kg = float(input(
        f"\nEnter Load (in kg) "
        f"[training range: {RANGES['Load(in kg)'][0]:.0f}"
        f"—{RANGES['Load(in kg)'][1]:.0f}]: "))

    print("\nTraffic Condition options: Low / Medium / High")
    traffic_condition = input("Enter Traffic Condition: ").strip().

    hard_throttle = float(input(
        f"\nEnter Hard Throttle instances "
        f"[training range: {RANGES['Hard Throttle'][0]:.0f}"
        f"—{RANGES['Hard Throttle'][1]:.0f}]: "))

    # — Validate categoricals —————
    traffic_map = {'Low': 0, 'Medium': 1, 'High': 2}
    if traffic_condition not in traffic_map:
        print("\n❌ Invalid Traffic Condition. Choose: Low, Medium,
        return

```

```

# — Out-of-range warnings —————
print()
any_warning = False
if not (RANGES['Load(in kg)'][0] <= load_kg <= RANGES['Load(in
    print(f"⚠ WARNING: Load {load_kg} kg is outside the tra
        f"({RANGES['Load(in kg)'][0]:.0f}–{RANGES['Load(in kg
        f"Prediction may be unreliable.")
    any_warning = True

if not (RANGES['Hard Throttle'][0] <= hard_throttle <= RANGES['
    print(f"⚠ WARNING: Hard Throttle {hard_throttle} is outs:
        f"({RANGES['Hard Throttle'][0]:.0f}–{RANGES['Hard Thr
        f"Prediction may be unreliable.")
    any_warning = True

if any_warning:
    confirm = input("\nProceed anyway? (yes/no): ").strip().low
    if confirm != 'yes':
        print("Prediction cancelled.")
        return

# — Build input dataframe —————
# Note: 'Condition of roads' dropped after Phase 2 analysis
input_df = pd.DataFrame(
    [[load_kg, traffic_map[traffic_condition], hard_throttle]],
    columns=['Load(in kg)', 'Traffic conditions', 'Hard Thrott
)

# — Scale using training scaler —————
input_scaled = scaler.transform(input_df)

# — Predict with all models —————
print("\n" + "="*55)
print("PREDICTED MILEAGE")
print("="*55)
for name, model in trained_models.items():
    pred = model.predict(input_scaled)[0]
    print(f" {name:30s}: {pred:.2f} km per litre")
print("="*55)

if any_warning:
    print("\n⚠ Note: One or more inputs were outside training
    print(" Treat predictions above with caution.")

# — Run —————
predict_mileage_interactive()
=====

```

HARLEY DAVIDSON X440 MILEAGE PREDICTOR

=====

Enter Load (in kg) [training range: 60–206]: 60

Traffic Condition options: Low / Medium / High

Enter Traffic Condition: medium

Enter Hard Throttle instances [training range: 3–16]: 15

=====

PREDICTED MILEAGE

=====

Linear Regression	: 21.11 km per litre
Ridge Regression	: 21.20 km per litre
Lasso Regression	: 20.82 km per litre
Random Forest	: 18.23 km per litre
Gradient Boosting	: 22.39 km per litre

=====